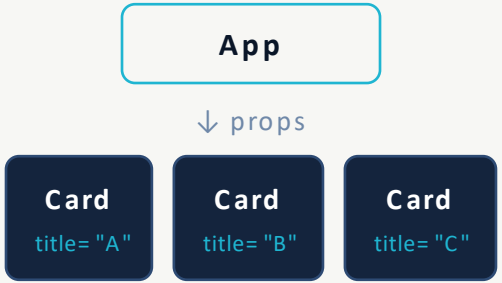
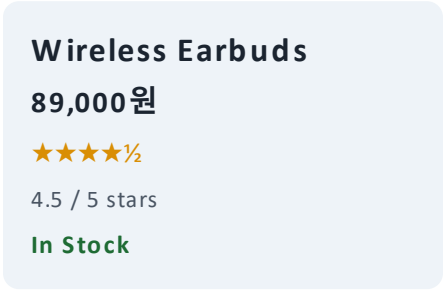
React Components & Props



Today's Agenda

- What is a React component?
- JSX: HTML inside JavaScript
- Props: passing data to components
- Composing components
- Lists & conditional rendering
- Code challenge: build a card component

By the end of class



A reusable card component

Part 1: What Is a Component?

A function…

```
function Greeting() {
return <h1>Hi</h1>;
}
```

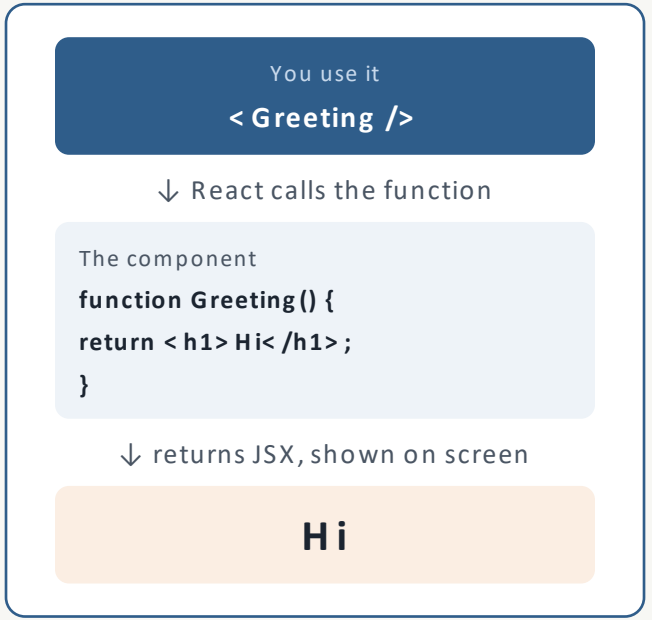


…that returns UI

Hi

Component = Function That Returns JSX

- **JSX is not HTML** compiler turns <h1>Hi</h1> into React.createElement("h1", null, "Hi")
- **Capital letter = component.** is an HTML tag, and <Greeting /> calls your function.
- **Re-renders on change.** When props or state change, React calls the function again.
- **Can return null** to render nothing.
- **Props flow down.** A parent passes data to a child, and the child shouldn't modify it.



Your First Component

```
// Simplest possible React component
function Greeting() {
  return <h1>Hello, World!</h1>;
}

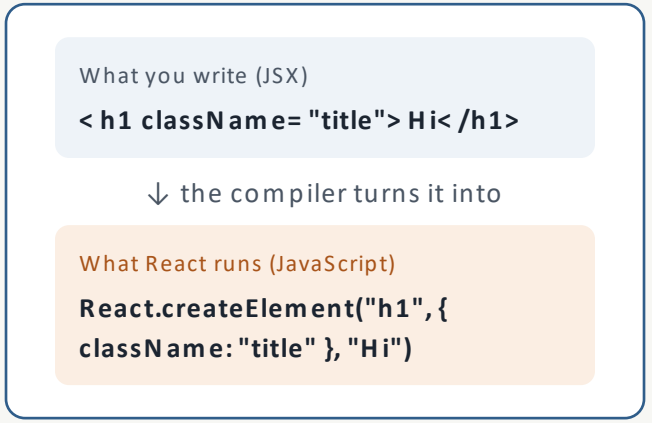
// Using it in another component
function App() {
  return (
    <div>
      <Greeting />
      <Greeting />
      <Greeting />
    </div>
  );
}
```

Hello, World!
Hello, World!
Hello, World!

https://muxiddin19.github.io/web_pro/week11/Hello,%20World!.html

JSX: JavaScript + XML

- **Not HTML.** It compiles to React.createElement()
- **className, not class.** class is a JavaScript keyword.
- **htmlFor, not for.** Same reason.
- **{ } for JavaScript.** Put any expression inside curly braces.
- **Close every tag.** Write <img />, not <img>.



```
function UserCard({ name, role }) {
  const initials = name.split(' ').map(w => w[0]).join('');

  return (
    <div className="card">
      <div className="avatar">{initials}</div>
      <h2>{name}</h2>
      <p className="role">{role}</p>
      <p>Joined: {new Date().getFullYear()}</p>
    </div>
  );
}
```

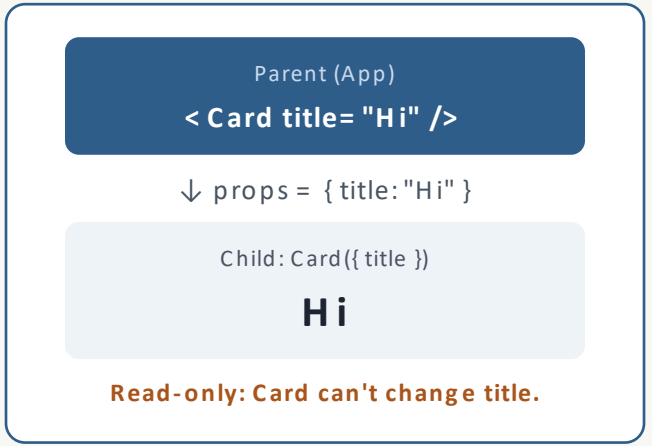


https://muxiddin19.github.io/web_pro/week11/User Cards.html

Part 2: Props — Passing Data Down

Props: Component Arguments

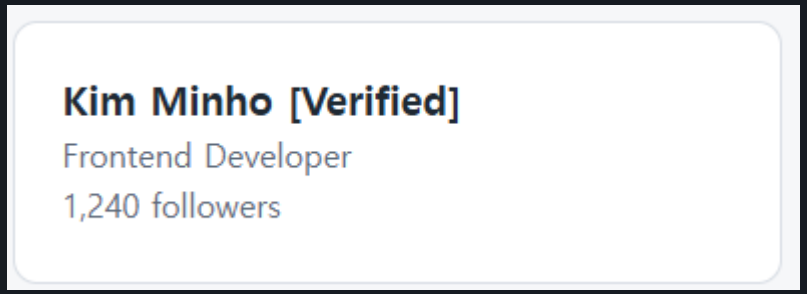
- **Passed like HTML attributes.** <Card title="Hi" />
- **Received as an object.** The component gets props, usually destructured: { title }.
- **Read-only.** A component never modifies its props.
- **Any JS value works.** Strings, numbers, arrays, objects, and functions.
- **Plan for missing values.** A prop can be undefined, so use defaults: { title = "Untitled" }.



Passing and Using Props

```
// Parent passes props
function App() {
  return (
    <ProfileCard
      name="Kim Minho"
      role="Frontend Developer"
      followers={1240}
      isVerified={true}
    />
  );
}

// Child receives and uses them
function ProfileCard({ name, role, followers, isVerified }) {
  return (
    <div className="profile">
      <h2>{name} {isVerified && '[Verified]'}</h2>
      <p>{role}</p>
      <p>{followers.toLocaleString()} followers</p>
    </div>
  );
}
```



https://muxiddin19.github.io/web_pro/week11/Profile Card.html

Default Props & PropTypes

- **Defaults handle missing props.** function Button({ label = "Click me", size = "md" }) { … }
- **PropTypes** (the prop-types package) adds runtime type checks.
- **TypeScript uses an interface** (or type) instead of PropTypes.
- **Always plan for undefined and null** in your JSX.

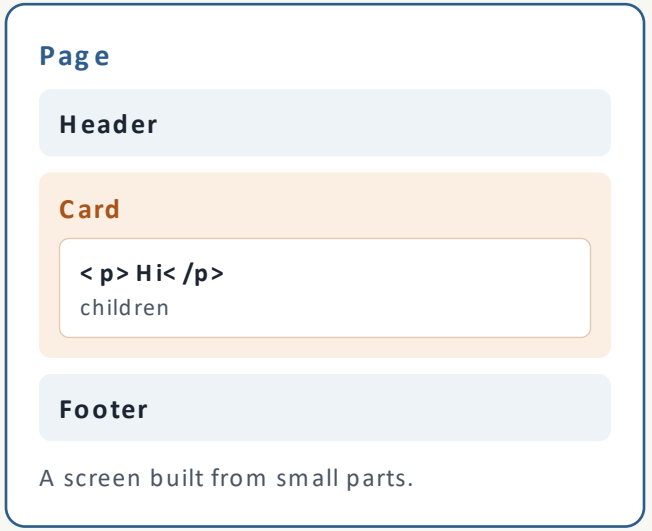


Part 3: Composing Components



Component Composition

- **Small, single-purpose components.** Each one does one job.
- **Compose them into larger ones.** Build screens from small parts.
- **Children let a component wrap others.** <Card> Hi </Card>
- **Think in React.** Each box in a mockup is one component.
- **Keep trees shallow.** Small, focused components are easier to test and debug.

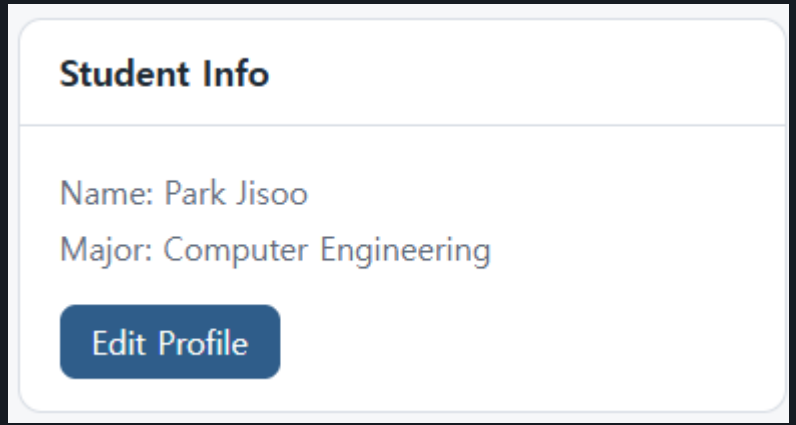


A screen built from small parts.

props.children: Wrapper Components

```
// A reusable card wrapper
function Card({ title, children }) {
  return (
    <div className="card">
      <h3 className="card-title">{title}</h3>
      <div className="card-body">
        {children}
      </div>
    </div>
  );
}

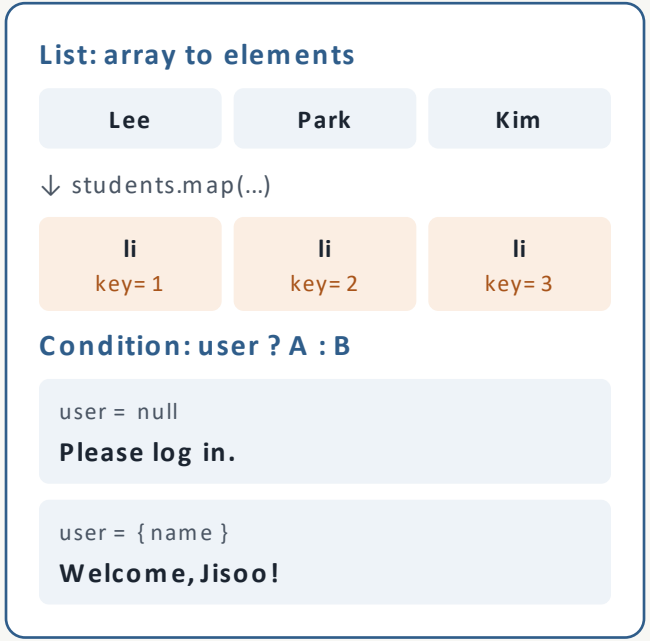
// Use it to wrap anything
function App() {
  return (
    <Card title="Student Info">
      <p>Name: Park Jisoo</p>
      <p>Major: Computer Engineering</p>
      <button>Edit Profile</button>
    </Card>
  );
}
```



https://muxiddin19.github.io/web_pro/week11/Card.html

Part 4: Lists & Conditional Rendering

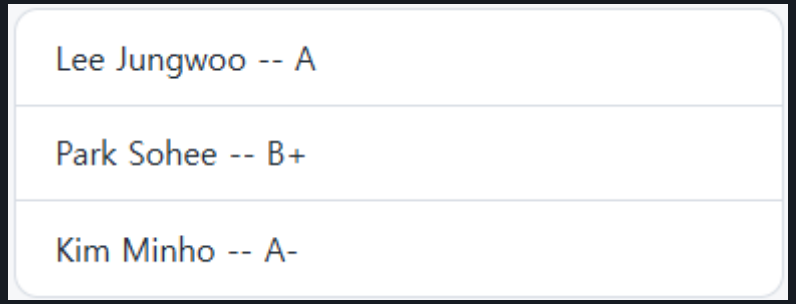
- Render a list with **.map()**: one element per array item
- Give every item a **unique, stable key** (an id, not the array index)
- Show or hide UI with **null, a ternary, or &&**
- A component can return **null** to render nothing



Rendering Lists with .map()

```
const students = [
  { id: 1, name: 'Lee Jungwoo', grade: 'A'  },
  { id: 2, name: 'Park Sohee',  grade: 'B+' },
  { id: 3, name: 'Kim Minho',   grade: 'A-' },
];

function StudentList() {
  return (
    <ul>
      {students.map(student => (
        <li key={student.id}>
          {student.name} -- {student.grade}
        </li>
      ))}
    </ul>
  );
}
```



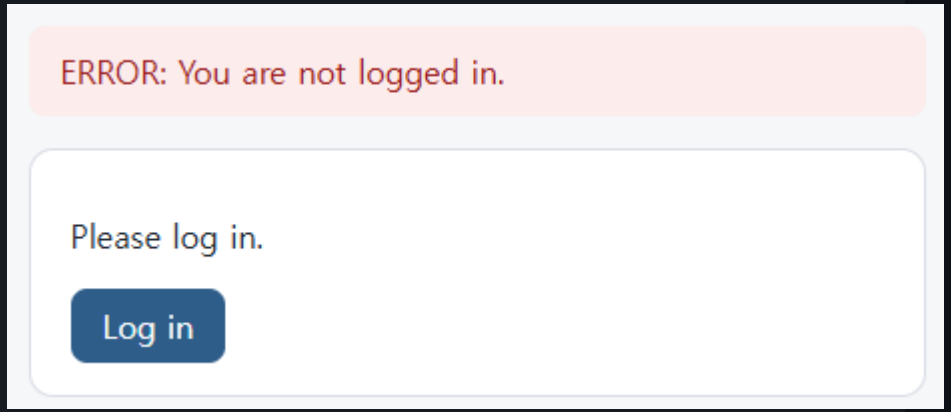
https://muxiddin19.github.io/web_pro/week11/Student List.html

Conditional Rendering

```
function Alert({ type, message }) {
  if (!message) return null; // render nothing

  return (
    <div className={"alert alert-" + type}>
      {type === 'error' ? 'ERROR:' : 'OK:'} {message}
    </div>
  );
}

function Dashboard({ user }) {
  return (
    <div>
      {user && <p>Welcome, {user.name}!</p>}
      {!user && <p>Please log in.</p>}
    </div>
  );
}
```



https://muxiddin19.github.io/web_pro/week11/Conditional Rendering.html

Component Best Practices

- **One component, one responsibility.** This is the Single Responsibility Principle.
- **Keep them small.** If a component does too much, split it.
- **Name them clearly.** ProductCard, not Comp1.
- **One file per component** in larger apps.
- **Default export** for the main component, named exports for utilities.

ProductCard.jsx

ProductImage
shows the photo

ProductTitle
shows the name

PriceTag
formats the price

BuyButton
handles the click

One job each: easy to test and reuse.

Code Challenge: Build a ProductCard

```
// Challenge: Build a ProductCard component
// that accepts these props:
//   name    (string)  — product name
//   price   (number)  — price in KRW
//   rating  (number)  — 0 to 5 stars
//   inStock (boolean) — is it available?

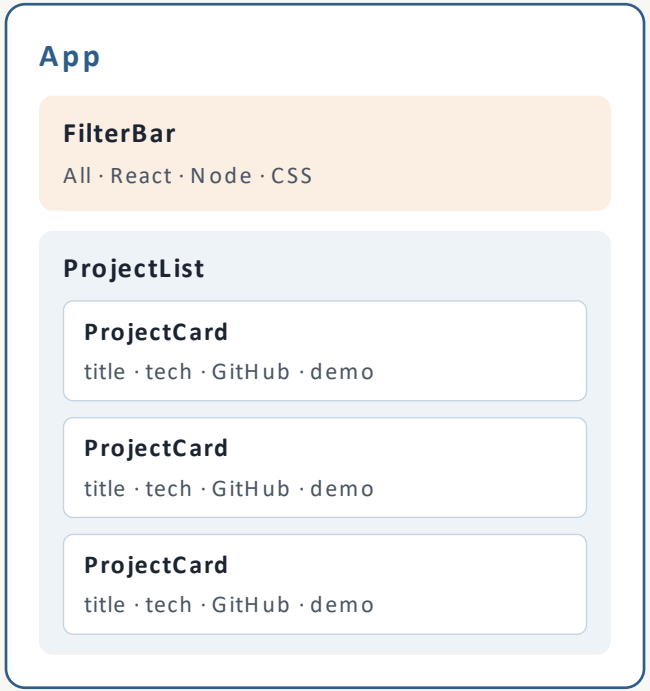
// Expected usage:
// <ProductCard
//   name="Wireless Earbuds"
//   price={89000}
//   rating={4.5}
//   inStock={true}
// />

// Bonus: display rating/5 (e.g., "4.5 / 5 stars")
// Bonus 2: show "Out of Stock" label if !inStock
```


https://muxiddin19.github.io/web_pro/week11/ProductCard.html

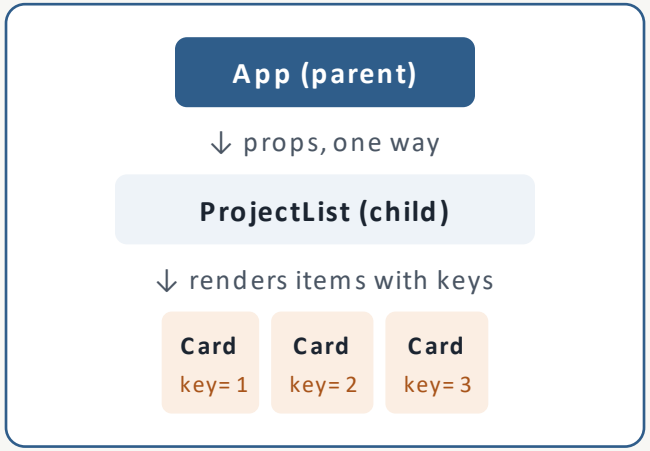
Homework: Portfolio Gallery Component

- Create a React project: **npx create-react-app portfolio-gallery**
- Build a **ProjectCard** component (title, description, tech stack, GitHub link, demo link)
- Build a **ProjectList** that renders an array of 3+ ProjectCards
- Add a **FilterBar** component that filters cards by technology
- Submit your GitHub repo URL on Cybercampus before next class



Key Takeaways

- Components are **functions that return JSX**
- Props flow **ONE WAY**: parent to child (never the reverse)
- Every list item needs a **unique, stable key** prop
- Conditional rendering: **null, ternary, or &&** operator
- **Compose** small components into larger ones: this is how React apps are built



Lecture notes: ReactProps_v1.pdf

_VideoLecture.html

VisualLecture2.html

StudyGuide.html

Weekly Quiz: viewform

Task_ReactProps.html

SprintQuiz.html

Next Session & Resources

- **Monday:** React State: components that remember things (useState Hook)
- **React official docs:** react.dev (new docs, excellent)
- **Scrimba React course** (free tier): scrimba.com/learn/learnreact
- **CodeSandbox:** codesandbox.io. Run React in the browser, no setup needed
- **Video:** React in 100 Seconds by Fireship on YouTube

